

An Analysis of Word2vec

Word2vec

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{j \neq 0} \log p(w_{t+j} | w_t)$$

Word2vec -- Part 1

Word2vec is a technique in natural language processing for obtaining vector representations of words. These vectors capture information about the meaning of the word based on the surrounding words. The word2vec algorithm estimates these representations by modeling text in a large corpus. Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence. Word2vec was developed by Tomáš Mikolov, Kai Chen, Greg Corrado, Ilya Sutskever and Jeff Dean at Google, and published in 2013. Word2vec represents a word as a high-dimension vector of numbers which capture relationships between words. In particular, words which appear in similar contexts are mapped to vectors which are nearby as measured by cosine similarity. This indicates the level of semantic similarity between the words, so for example the vectors for walk and ran are nearby, as are those for "but" and "however", and "Berlin" and "Germany".

Word2vec -- Part 2

Training algorithm A Word2vec model can be trained with hierarchical softmax and/or negative sampling. To approximate the conditional log-likelihood a model seeks to maximize, the hierarchical softmax method uses a Huffman tree to reduce calculation. The negative sampling method, on the other hand, approaches the maximization problem by minimizing the log-likelihood of sampled negative instances. According to the authors, hierarchical softmax works better for infrequent words while negative sampling works better for frequent words and better with low dimensional vectors. As training epochs increase, hierarchical softmax stops being useful.

Word2vec -- Part 3

History During the 1980s, there were some early attempts at using neural networks to represent words and concepts as vectors. In 2010, Tomáš Mikolov (then at Brno University of Technology) with co-authors applied a simple recurrent neural network with a single hidden layer to language modelling. Word2vec was created, patented, and published in 2013 by a team of researchers led by Mikolov at Google over two papers. The original paper was rejected by reviewers for ICLR conference 2013. It also took months for the code to be approved for open-sourcing. Other researchers helped analyse and explain the algorithm. Embedding vectors created using the Word2vec

algorithm have some advantages compared to earlier algorithms such as those using n-grams and latent semantic analysis. GloVe was developed by a team at Stanford specifically as a competitor, and the original paper noted multiple improvements of GloVe over word2vec. Mikolov argued that the comparison was unfair as GloVe was trained on more data, and that the fastText project showed that word2vec is superior when trained on the same data. As of 2022, the straight Word2vec approach was described as "dated". Transformer-based models, such as ELMo and BERT, which add multiple neural-network attention layers on top of a word embedding model similar to Word2vec, have come to be regarded as the state of the art in natural language processing.

Word2vec -- Part 4

Dimensionality Quality of word embedding increases with higher dimensionality. But after reaching some point, marginal gain diminishes. Typically, the dimensionality of the vectors is set to be between 100 and 1,000.

Word2vec -- Part 5

Sub-sampling High-frequency and low-frequency words often provide little information. Words with a frequency above a certain threshold, or below a certain threshold, may be subsampled or removed to speed up training.

Word2vec -- Part 6

Radiology and intelligent word embeddings (IWE) An extension of word vectors for creating a dense vector representation of unstructured radiology reports has been proposed by Banerjee et al. One of the biggest challenges with Word2vec is how to handle unknown or out-of-vocabulary (OOV) words and morphologically similar words. If the Word2vec model has not encountered a particular word before, it will be forced to use a random vector, which is generally far from its ideal representation. This can particularly be an issue in domains like medicine where synonyms and related words can be used depending on the preferred style of radiologist, and words may have been used infrequently in a large corpus. IWE combines Word2vec with a semantic dictionary mapping technique to tackle the major challenges of information extraction from clinical texts, which include ambiguity of free text narrative style, lexical variations, use of ungrammatical and telegraphic phrases, arbitrary ordering of words, and frequent appearance of abbreviations and acronyms. Of particular interest, the IWE model (trained on the one institutional dataset) successfully translated to a different institutional dataset which demonstrates good generalizability of the approach across institutions.

An Analysis of Word2vec

Word2vec

Word2vec -- Part 7

The idea of CBOW is to represent each word with a vector, such that it is possible to predict a word using the sum of the vectors of its neighbors. Specifically, for each word w_i in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w_i . The objective of training is to maximize $\sum_i \ln \Pr(w_i | w_{i+j} : j \in N)$ where N is a set of (non-zero) indices representing the relative locations of nearby words considered to be in w_i 's neighborhood. For example, if we want each word in the corpus to be predicted by every other word in a small span of 4 words. The set of relative indexes of neighbor words will be: $N = \{-2, -1, +1, +2\}$, and the objective is to maximize $\sum_i \ln \Pr(w_i | w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$. In standard bag-of-words, a word's context is represented by a word-count (aka a word histogram) of its neighboring words. For example, the "sat" in "the cat sat on the mat" is represented as {"the": 2, "cat": 1, "on": 1}. Note that the last word "mat" is not used to represent "sat", because it is outside the neighborhood $N = \{-2, -1, +1, +2\}$. In continuous bag-of-words, the histogram is multiplied by a matrix V to obtain a continuous representation of the word's context. The matrix V is also called a dictionary. Its columns are the word vectors. It has D columns, where D is the size of the dictionary. Let d be the length of each word vector. We have $V \in \mathbb{R}^{d \times D}$. For example, multiplying the word histogram {"the": 2, "cat": 1, "on": 1} with V , we obtain $2v_{\text{the}} + v_{\text{cat}} + v_{\text{on}}$. This is then multiplied with another matrix V' of shape $R \times D \times d$. Each row of it is a word vector v' . This results in a vector of length D , one entry per dictionary entry. Then, apply the softmax to obtain a probability distribution over the dictionary. This system can be visualized as a neural network, similar in spirit to an autoencoder, of architecture linear-linear-softmax, as depicted in the diagram. The system is trained by gradient descent to minimize the cross-entropy loss. In full formula, the cross-entropy loss is: $-\sum_i \ln \Pr(w_i | \sum_{j \in N} v_{w_{i+j}}) = -\sum_i \ln \left(\frac{e^{v_{w_i} \cdot (\sum_{j \in N} v_{w_{i+j}})}}{\sum_{w' \in \text{dictionary}} e^{v_{w_i} \cdot v_{w'}}} \right)$

$N \} v_{w_{i+j}}) = -\sum_i \ln \left(\frac{e^{v_{w_i} \cdot (\sum_{j \in N} v_{w_{i+j}})}}{\sum_{w' \in \text{dictionary}} e^{v_{w_i} \cdot v_{w'}}} \right)$ where the outer summation $\sum_i$ is over the words in a corpus, the quantity $\sum_{j \in N} v_{w_{i+j}}$ is the sum of a word's neighbors' vectors, etc. Once such a system is trained, we have two trained matrices V, V' . Either the column vectors of V or the row vectors of V' can serve as the dictionary. For example, the word "sat" can be represented as either the "sat"-th column of V or the "sat"-th row of V' . It is also possible to simply define $V' = V^T$, in which case there would no longer be a choice.